

南開大學
Nankai University

南　开　大　学

计 算 机 学 院

操作系统第三次实验报告

中断与异常处理

Trap 现场恢复及定时中断

组长：　李珉熠　2413982

组员：　吴宇石　2411080

组员：　赵亚鑫　2413715

2026 年 11 月 2 日

摘要

我们在 Lab2 基础上完成中断与异常处理机制，包括 Trap 入口、现场恢复、定时事件重新设置，以及非法指令和断点处理。课程三项评分分别为 35、25 和 40 分，实测总分 100/100。报告说明实现约束、提示词、真实编译错误和验证范围。

关键词：Trap；上下文；定时中断；异常；RISC-V

目录

一、 基本信息与分工 **1**

二、 目的、环境与整体逻辑 **1**

三、 Lab2 基础与启动兼容 **1**

四、 模块一：Trap 入口和现场恢复 **1**

五、 模块二：中断与异常处理 **2**

（一） 时钟中断 2

（二） 非法指令与断点异常 2

六、 Spec 与两份 Prompt **3**

（一） 模块一 Spec 与 Prompt 3

（二） 模块二 Spec 与 Prompt 3

七、 实际排错与验证 **3**

八、 原理联系、局限和总结 **4**

一、 基本信息与分工

实验日期：2026-11-02。我们基于课程 lab3.zip 完成实现，使用 Codex 辅助分析、实现、排错和文档整理，以实际编译、QEMU 运行及 GDB 观察核对结果。以下为小组模块分工。

成员	模块
李珉熠 2413982（组长）	环境、Lab2 移植、构建与报告汇总
吴宇石 2411080	Trap 入口、汇编恢复和布局分析
赵亚鑫 2413715	中断异常处理、评分与调试分析

二、 目的、环境与整体逻辑

我们使用 Ubuntu 虚拟机、RISC-V GCC 15.1.0、QEMU 8.2.2 和 OpenSBI 1.3，实验目录为/home/lmy/os/lab3。

Trap 是中断与异常的统称。中断是与当前指令执行异步的事件，例如定时器到期；异常由当前指令执行引发，例如非法指令或断点。上下文是被打断时的寄存器状态，trapframe 是保存在栈上的现场结构。

整体顺序如下：

```
初始化DTB和物理内存 → 设置stvec统一入口
→ 设置时钟事件和中断使能 → 发生Trap
→ __alltraps / SAVE_ALL → trap(tf) → trap_dispatch
→ interrupt_handler或exception_handler
→ __trapret / RESTORE_ALL → sret返回原执行流
```

stvec 保存 S 态 Trap 入口地址；S 态为监管者模式，即本次内核运行的权限级别。硬件设置 sepc、scause 及相关状态，通用寄存器则由入口汇编保存，不能认为硬件自动完成全部现场保存。

三、 Lab2 基础与启动兼容

我们移植 page_init 和 Best-Fit，保留 Lab3 中 alloc_pages、free_pages 和 nr_free_pages 的局部中断保护包装。分配前记录原中断状态并关闭本核中断，操作后恢复原状态，不能无条件重新开中断。这避免本核中断处理打断链表修改，但不提供多核互斥。

物理内存通过 DTB 取得，描述符与保留区沿用 Lab2 设计。我们补充 PHYSICAL_MEMORY_END=0x88000000、KERNEL_BEGIN_PADDR=0x80200000 两个移植需要的常量。QEMU 加载改用-kernel，启动根表增加低地址过渡映射；工具链明确-march=rv64gc、-mabi=lp64d，并使用 LF 换行。

四、 模块一：Trap 入口和现场恢复

我们补全 trapentry.S 中的 RESTORE_ALL 及 ___alltraps 调用部分，保留 SAVE_ALL 布局、trapframe 结构和 trap 函数签名。

NKU

RV64 下每个槽位 8 字节，32 个通用寄存器槽位加 4 个控制状态槽位，共 36×8=288 字节。控制状态依次是 sstatus、sepc、stval 的保存值和 scause。CSR 是控制与状态寄存器；stval 提供异常附加信息，scause 记录原因，sepc 记录返回执行位置。

SAVE_ALL 先把原 sp 放到 sscratch，再将 sp 下移 288 字节，原 sp 最终保存在第 2 号槽位。sp 是栈指针，sscratch 是辅助暂存 CSR。因此我们先读出并恢复 sstatus、sepc，再恢复通用寄存器，最后恢复 sp，保证所有现场读取仍以当前 frame 为基址。cause 和 badvaddr 供处理与诊断，不必像 sepc 一样写回才能返回。

___alltraps 用 mv a0,sp 将 frame 指针作为第一个参数，call trap 调用 C 函数，函数返回后进入 ___trapret。a0 是 RISC-V 调用约定中的首参数寄存器，ra 保存函数返回地址；旧 a0、ra 已在 frame 中，最终一并恢复。

sret 是 S 态 Trap 返回指令，根据 SPP 恢复特权级、根据 SPIE 恢复中断使能，并将 sepc 作为继续执行的位置。SPP 为之前的权限级别，SPIE 为之前的中断使能状态。本实验主要由 S 态返回 S 态，不是每次都返回用户态。

五、 模块二：中断与异常处理

（一） 时钟中断

trap_dispatch 通过 cause 最高位区分中断和异常，我们在 interrupt_handler 中清除该位后按原因分类。S 态时钟原因编号为 5。

每次时钟中断调用 clock_set_next_event，ticks 加 1；当 ticks 是 TICK_NUM=100 的整数倍时调用既有 print_ticks。clock_set_next_event 通过 SBI 接口设置未来事件；SBI 是内核请求 M 态固件服务的接口。timebase=100000 配合 10MHz 时间计数器，约每 10ms 设置下一次事件，实际间隔受处理开销影响，不能等同精确 CPU 周期。

指导书引用 SHUTDOWN_AFTER，但下载框架未给出定义或明确数值。我们在允许实现区域定义可覆盖默认值 10，含义是打印 10 次 100 ticks 后调用 sbi_shutdown，而非声称老师规定必须 10 次。正常运行预期 1000 次中断后关机；时钟评分在 200 次时已验证持续返回。

其他软件、定时和外部中断按类型输出提示；未知原因打印编号及 frame。外部中断来自设备，软件中断用于软件请求，本实验不实现完整设备中断应答协议。

（二） 非法指令与断点异常

我们识别 CAUSE_ILLEGAL_INSTRUCTION 和 CAUSE_BREAKPOINT，输出 Illegal instruction at 或 Breakpoint at。读取 epc 指向指令的低 16 位：低 2 位不全为 1，按 2 字节跳过；否则低 5 位不全为 1，按 4 字节跳过；更长编码在本教学实现中停止并报告不支持，不猜测跳过。

这适用于本实验 RV64GC 的 16/32 位指令。若不修改 epc，返回后会再次执行同一异常指令，形成循环。异步时钟中断不主动增加 epc，以免跳过正常代码。其他异常按框架要求打印 frame，不宣称已经恢复缺页等故障；当前读取故障指令的方式也不适用于任意不可访问 epc。

六、 Spec 与两份 Prompt

我们将本次协作需求整理为下面两份提示，AI 作为辅助，生成结果通过框架约束和实测核对。

（一） 模块一 Spec 与 Prompt

Spec：输入现有 SAVE_ALL 和 36 槽位 frame；输出与其严格对应的恢复汇编，以及将 frame 传入 trap 的调用逻辑；不得修改保存布局、结构体或函数签名。

Prompt：阅读 trap.h、trapentry.S 和 RISC-V 调用约定，仅补全 RESTORE_ALL 和 ___alltraps 留空处。逐项列出 32 个通用寄存器及 4 个状态槽位的偏移，识别原 sp 经 sscratch 保存到第 2 槽位的特殊关系。先恢复返回需要的 CSR 和通用寄存器，最后恢复 sp，避免破坏 frame 访问基址。以 a0 传入当前 frame，调用 trap 后接入既有 ___trapret 和 sret；保持 16 字节栈对齐。解释每步为何与 SAVE_ALL 对应，并保留 lab3 注释、替换学号标记。

（二） 模块二 Spec 与 Prompt

Spec：接收 frame，根据原因分类；定时器持续重设和计数；异常按实际 16/32 位长度推进 epc；保留其余框架函数和课程评分条件。

Prompt：在 trap.c 规定区域实现 interrupt_handler 和 exception_handler。时钟事件每次重设，ticks 每次加 1，每 100 次使用已有函数输出，定义并说明可配置关机次数默认值，保证评分可到 200 次。其他中断分类输出，未知原因打印 frame。非法指令和断点按要求输出地址，通过指令低位编码判断 2/4 字节长度后推进 epc，未知异常仅打印现场，异步中断不增加 epc。使用现有 sbi_shutdown；不要修改 trap_dispatch、trap 签名、SAVE_ALL 或课程测试。

七、 实际排错与验证

我们编译时实际遇到两处错误：当前汇编器拒绝旧 CSR 名 sbadaddr，因此在汇编文件头用预处理别名映射到同一 CSR 的现代名称 stval，保持 SAVE_ALL 主体不变；移植 page_init 后缺少两个内存布局常量，核对 Lab2 与 Lab3 映射后补充。启动加载参数和过渡映射问题则是根据前次实验经验预先适配，没有将其写成本次已经出现的运行故障。

课程测试源码和注入脚本保留原样，评分脚本只适配 QEMU 加载方式，未改变通过条件及分数。

验证	结果	能支持的结论
现场恢复及断点返回	35/35	课程预置的多组寄存器经 ebreak 后保持正确
非法指令及兜底处理	25/25	4 字节非法指令可返回，软件中断与未知原因输出符合要求
持续时钟	40/40	初始化计数、重新设定和反复返回支持运行到 200 次
总分	100/100	本次课程三项检查均通过
普通启动	10 次 100 ticks 后正常退出	非评分构建的时钟与关机路径可运行

验证	结果	能支持的结论
客体 GDB	frame=288 字节，cause=0x8000000000000005	首次时钟进入 trap 时 ticks=0，之后进入 ___trapret

完整的 16 位非法指令、全部扩展指令、任意嵌套异常和用户态栈切换不在上述评分覆盖中，不能凭 100 分推断全部正确。我们将实际启动和 GDB 记录留在本地，GitHub 只保留代码与报告。

八、 原理联系、局限和总结

本实验中的现场保存与恢复对应操作系统的上下文管理，但尚未切换进程；时钟提供调度的基础事件，但本次没有进程调度器；非法指令处理体现同步异常分发，但按需分页、页面置换和用户系统调用尚未实现。关闭本核中断保护临界区不等于多核锁，volatile 使 ticks 按要求进行可见访问，也不等于原子读改写。

通过本次实验，我们接通了从汇编入口、C 处理函数到 sret 返回的完整链路，理解了硬件记录控制状态与软件保存通用寄存器的分工。

参考：课程练习、Trap 入口、时钟、报告要求。

NKU